Agent Name	System Metric	Testing Methodologies				CI/CD Pipeline Practices				Code Review Process				Bug Solving Process				SE Methodology				Developer Communication				Hollistic Documentation			
	Description	Practices	Tools	Stakeholders	Others	practices	Tools	Stakeholders	Others	practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others
Hubspot breeze Studio	Closed-source HubSpot Breeze studio is a CRM-integrated AI agent creation platform in sales, marketing, and customer service with various agents. Java (MCP server, backend infrastructure), TypeScript (developer tools), JavaScript (UI components) [q-1]	HubSpot's testing practice is to use React Testing Library as its official UI testing framework. They emphasize user-focused tests over implementation details. In the testing migration, they collected runtime usage metrics across the team and then categorized packages into buckets. The company migrated more than 76,000 tests from Enzyme to React Testing Library and runs frontend tests in the browser rather than JSDOM [q-2] Migration Story: The HubSpot testing team adopts the Testing Trophy strategy when embracing React Testing Library. The Testing Trophy is a testing strategy that prioritizes integration tests as the best balance between confidence and maintenance cost, while still using static, unit, and end-to-end tests appropriately. At first, they documented guides for writing tests with common libraries and developed tools to minimize test fixtures. By adopting this philosophy, teams felt	React testing library, Monarch, Crucible, Claude Code, TypeScript [q-7] [q-8]	developers [q-9]	The testing migration spanned 2.5 years with 76,000 tests migrated, peaking at 4,000+ tests per month. Migration was completed in February 2025 using runtime metrics instrumentation rather than static analysis. [q-10] CD adoption required ~9 months and ~20 new test suites across 4 repositories. HubSpot deploys up to 100 times per day. [q-11]	HubSpot operates a Kubernetes-based CI/CD system, running over 1 million builds per day across 3,000 EC2 instances that host tens of thousands of microservices. [q-12] Cloud coding agents use Kubernetes Jobs with custom Docker images that contain Claude Code and developer tooling preinstalled. [q-12] The team reduced slow Java build times by running build agents on a dedicated Kubernetes node pool and reusing a host-mounted build cache, which cuts build times from 10–15 minutes to 2–3 minutes. [q-13] The platform leverages enterprise-grade features like auto-scaling, observability, and region-aware API routing via Cloudflare Workers. [q-13] Note: HubSpot- company-specific, not specific,	An internal build and deploy system built on top of Kubernetes [q-15] Orion (HubSpot deploy system) [q-16]	Infrastructure teams, platform engineering teams, and Developer Experience AI team [q-19]	They build an internal platform for executing cloud coding agents [q-17] [q-18]	HubSpot uses Sidekick AI code reviewer with Judge Agent quality gate. [q-20] Their AI code reviewer catches real issues, understands HubSpot-specific context, and maintains a high signal to noise ratio. [q-20] After Sidekick drafts a review, the agent receives that output and evaluates it against three core criteria: Succinctness: Is the feedback clear and to the point? Accuracy: Is the suggestion technically correct within the context of the codebase and change? Actionability: If the feedback includes a code suggestion, can it be applied directly without additional	Sidekick (AI code review bot), Judge Agent for quality filtering, Aviator (internal Java-based agent framework that replaced Crucible), multiple AI models (Claude, GPT, Gemini), GitHub CLI, and emoji-based	developers [q-23]	Sidekick has reviewed 50,000 + pull requests with 90% time-to-feedback reduction and 80%+ approval rate. The Judge Agent filters reviews based on succinctness, accuracy, and actionability criteria before posting. [q-51]	HubSpot's bug solving process relies on rapid tracing and automated rollbacks when standard monitoring fails. During the Feb 2026 incident, standard health checks failed to detect a permissions error because endpoints returned successful HTTP 200 responses with incorrect data. The incident was declared 16 minutes after full rollout due to customer support surges, and engineers resolved it via a full automated rollback within 37 minutes total. [q-24] Users can easily trigger a planning step on a GitHub issue by directly mentioning our internal bot ("@SidekickAI create a plan").	comprehensive metrics dashboards for token/API cost monitoring, Monarch for migration campaign tracking, and emoji reaction feedback for continuous improvement of AI agents [q-68]	Infrastructure Team, reliability Engineer [q-69]	HubSpot Breeze is closed-source with no public issue tracker. The Feb 2026 incident revealed that HTTP 200 responses with incorrect data bypassed standard error-rate monitoring — prompting a shift to response-correctness metrics. [q-70]	They prioritize rapid execution over perfect planning, moving from ideation to shipped product in hours or days. Work is driven by small, highly focused units of 3-4 engineers. Technical leaders act as hands-on, active contributors rather than traditional managers. Teams prioritize shipping quickly over perfect planning, with calculated risk-taking and low bureaucracy tolerance. [q-72] AI fluency is a mandatory baseline for all engineers. They have deeply embedded AI agents into their daily workflows, PRs, and code reviews,	Crucible Agent [q-75]	Tech leads, Engineering leads, managers [q-76]	They move fast and take calculated risks. [q-77]	HubSpot uses emoji reactions (like, dislike) and thread replies for feedback on code reviews, Team culture emphasizes psychological safety, mutual support, and continuous feedback. They frequently use Discord in the communication phase [q-36] [q-37]	Github,Discord, and slack [q-38]	All HubSpot engineers participate in team communication, supported by Developer Experience AI team, product teams, and engineering leadership fostering psychologically safe environment. [q-78]	Communication philosophy prioritizes reducing friction by meeting users in their existing tools (GitHub, Slack) rather than requiring new platforms. Feedback culture is continuous with emphasis on psychological safety and constant improvement. [q-79]	HubSpot offers two distinct MCP servers (one for CRM access, one for Developer Platform) with comprehensive documentation at developers.hubspot.com/mcp. [q-39] [q-40]	no specific tools mentioned [q-41]	developers [q-42]	Documentation strategy includes two MCP servers (remote CRM access + local Developer Platform), comprehensive NPM package documentation, testing guides for common libraries, EPIC Week annual conference for knowledge sharing, and example-driven documentation optimized for AI model integration. [q-43]
WatsonX Orchestrate	IBM Watsonx Orchestrate is an enterprise AI workflow orchestration platform that enables organizations to build, deploy, and manage AI agents automating business processes across domains such as HR, IT, and finance. Programming language: Python. The core WatsonX Orchestrate platform itself is closed-source; only the Agent Development Kit (ADK) is publicly available. Update Frequency: 1 bi-weekly release, 61 contributors, 153 stars (ADK). [q-49]	The IBM Watsonx Orchestrate Agent Development Kit (ADK) is a set of tools designed to make it easy to build and deploy agents using IBM Watsonx Orchestrate. Contributors run, build, and test the ADK locally with new feature, enhancement. After that, they can connect to a production instance of watsonx orchestrate. [q-2] Build-Time Evaluation Experience — a simulation engine that lets builders run hundreds of synthetic conversations against an agent from within the UI before any real user sees it. [q-3] The developer can navigate a structured execution timeline and inspect the input, reasoning, and output at each individual step in the Debug Experience. [q-5] The ADK ships a built-in red-teaming command group that simulates 15+ adversarial attack types against an agent. (prompt injection, instruction override). Builders run a subset of attacks first to validate their setup, then expand for broader variant coverage. [q-6] The evaluation framework in the ADK is designed to assess agent behavior by comparing simulated agent interactions, referred to as trajectories, against a predefined set of reference data. [q-50]	Hatch (Python testing tool), OpenTelemetry (OTEL) spans power the Debug Experience and trace view [q-51]	Product engineers. [q-9]	The developers define and assess success criteria such as expected tool calls with desired parameters and final response quality [q-10]	Average 2 minor releases per month, The Developer Edition ships as a Docker Compose application managed through the `orchestrate server` CLI. [q-11] [q-12] [q-13]	Github release, Docker Compose [q-14]	Internal developers [q-16]	The 2.0.0 release in November 2025 was the most structurally disruptive, requiring users to delete their entire local Developer Edition install (docker system prune -a --volumes) before upgrading. The team removed all default credentials for Developer Edition services and hardened the default security posture — a deliberate choice to eliminate shared-secret anti-patterns even at the cost of a one-time migration burden for existing developers. [q-15]	GitHub pull requests are the main code review channel. Every change, whether it is a security fix, a documentation update, or a new release; requires at least one peer approval before it can be merged. [q-17] Commit messages follow a consistent structured format that identifies what type of change was made and which part of the system it affected. This makes it straightforward to track what changed, generate release notes automatically, and separate documentation updates from functional changes. [q-18]	GitHub pull requests [q-20]	specific team members designated as owners for different parts of the codebase [q-21]	The code review process is intentionally lightweight — one approval, clear ownership, and consistent commit message formatting. This keeps the review process fast without sacrificing accountability, allowing the team to ship updates frequently. [q-53]	Issues resolved by the team fall into five recurring categories: (1) Environment and connectivity (2) Agent configuration and behavior (3) Data import and export (4) Authentication (5) UI inconsistencies [q-22] The IBM community forum functions as an external bug-signal channel: builders report issues that the engineering team monitors and typically addresses in the next release cycle. Known issues are explicitly called out in release notes. [q-23] Customers of watsonx Orchestrate can raise issues with the ADK or the Developer Edition server to IBM Support {Quote 3} [q-23]	IBM Support portal for enterprise customer tickets [q-24]	Core Maintainers [q-25]	Not Available	The ADK documentation formalizes a four-stage iterative development lifecycle that every builder follows before deploying an agent to production: (1) Develop — build and configure the agent using the ADK CLI and Python tools; (2) Evaluate — run the trajectory evaluation suite against a ground-truth dataset to measure Journey Success; (3) Analyze — inspect failing trajectories to identify whether bugs lie in tool definitions, agent instructions, or parameter routing; (4) Improve - update the relevant artifact, re-run evaluations, and watch the score change. This loop is designed to repeat as many times as needed within the Developer Edition before production deployment ever occurs [q-26] Developer tooling investment, like enhancing VS Code extension features, agent communication directly from CLI, reflects a philosophy of eliminating friction at every stage of the engineering loop.	The ADK added Agent-to-Agent (A2A) protocol support, enabling external agents to communicate natively with watsonx Orchestrate agents through a standardized interface. [q-30]	The methodology is shaped by a 61-contributor team distributed across IBM's enterprise AI organization. Flow-builder specialists own the workflow authoring subsystem. Community feedback from 461 forum threads shapes the product roadmap through the Gustavo Villegas Technical Touchpoint newsletter. [q-31]	Not Available	The most visible form of internal team coordination is through GitHub itself. Pull request descriptions, reviewer assignments via module ownership, and structured commit messages serve as the team's asynchronous coordination layer [q-32] [q-33] [q-34]	Github pull request is the primary source IBM's internal communication platforms (such as Slack, Microsoft Teams, or internal wikis) are not publicly disclosed. [q-35]	Internal coordination involves the module owners defined in CODEOWNERS, the release engineers who consolidate multi-squad contributions, and the feature engineers who author PRs. IBM does not publicly disclose whether it uses engineering managers, scrum masters, or designated leads to facilitate internal team communication. [q-57]	The team does maintain external communication channels — a monthly community newsletter, a public community forum, and live Ask Me Anything sessions tied to major product events. These channels are IBM-to-community communication, not internal team communication. IBM's internal communication practices (meetings, design docs, RFCs, chat tools) are not publicly available. [q-58]	The IBM team maintains a comprehensive developer documentation portal at developer.watson-orchestrate.ibm.com that covers the full builder journey end to end: ADK CLI reference, Developer Edition setup, agent authoring patterns, evaluation framework, and red-teaming. [q-36] [q-37] [q-38] [q-39]	Documentation stack: developer.watson-orchestrate.ibm.com (primary ADK developer docs), IBM Docs at ibm.com/docs/en/watsonx/watson-orchestrate (official product documentation), GitHub Releases (technical changelog and migration guides), IBM community forum (126 practitioner blogs) [q-40]	Documentation is maintained by a distributed group: Darragh-McGonigle and Alex Haynes own the README; @Eric-Grehan and @erifsx manage CODEOWNERS and release notes; Bruno Fernandes Dos Santos, Kelly To, and Alex Straley author feature announcement blog posts in the IBM community; and the IBM technical writing team handles IBM Docs and IBM Style compliance. All documentation changes flow through the same PR review gate as code. [q-41]	The ADK developer documentation spans multiple content types: conceptual guides, CLI reference pages, how-to walkthroughs, evaluation framework schema definitions, red-teaming attack catalog, and per-version release notes. As of May 2026, a dedicated ADK release notes page (developer.watson-orchestrate.ibm.com/release/release) documents the full feature and bug-fix history from 2.0.0 through the current release, making it the authoritative changelog for builders who need to understand what changed between versions. [q-42]
Gemini Enterprise	Google Gemini Enterprise (also marketed as Google Agentspace) is a closed-source enterprise AI agent platform built on Google Cloud that enables organizations to deploy and orchestrate intelligent AI agents for workplace knowledge retrieval, multi-step workflow automation, and enterprise productivity at scale. It offers prebuilt connectors for Confluence, Jira, SharePoint, and ServiceNow, and includes Agent Designer for building custom agents. Deployed as a SaaS on Google Cloud with no public source repository. [q-63]	Gemini Enterprise uses a staged feature lifecycle — Private Preview → Public Preview → GA (General Availability)— as a quality gate before features reach all users. Agent Designer went from Private Preview (April 2025) to GA (January 2026), a 9-month staged rollout. [q-1] SLO (Service Level Objective) monitoring is used to assess model quality for newly launched model tiers. After releasing Gemini 3.1 Pro and 3 Flash in Limited Availability, the team explicitly stated it was determining long-term SLOs, indicating SLO-based quality gates for model releases. [q-2] Administrator-facing observability tooling (Metrics Explorer, Trace Explorer) allows monitoring of platform quality metrics and distributed traces in production. [q-3] [Google-wide, not confirmed Gemini Enterprise-specific]: Google's Test Automation Platform (TAP) handles 50,000+ unique changes and 4 billion test cases per day. Hermetic testing (self-contained environments with no external dependencies) reduces instability. [q-39]	Metrics Explorer and Trace Explorer (Google Cloud Observability) for admin-facing quality monitoring in production. [q-4] [Google-wide, not confirmed Gemini Enterprise-specific]: TAP (Test Automation Platform) for CI/CD, Forge for distributed build execution across Google's datacenters. [q-39]	Not Available — Gemini Enterprise is closed-source.	Not Available — Additional evaluation details are not publicly disclosed. {Others}	Gemini Enterprise practices continuous deployment to production, evidenced by near-daily public release notes since January 2025. [q-42] Product was formally renamed from Google Agentspace to Gemini Enterprise on October 9, 2025, signaling milestone-based product management. [q-44] [Google-wide]: Release automation assembles release candidates (RCs) from green head. Any change to Google's codebase can be rolled back with two clicks. Feature flags and experiments are used for selective continuous deployment. [q-45] [Google-wide]: Large-scale automated code changes across the monorepo use the Rosie tool, which splits changes into separate CLs and sends them out for automated review with presubmit checks. [q-46]	[Google-wide, not confirmed Gemini Enterprise-specific]: TAP (Test Automation Platform) for CI, Forge for distributed build execution, Rosie for large-scale automated changes across the monorepo. Specific Gemini Enterprise CI/CD tooling is Not Disclosed. [q-47]	At Google, each team assigns a “Build Cop” who monitors the build system, ensures all tests pass, and rolls back changes if something breaks. Gemini Enterprise-specific DevOps/SRE team structure is Not Disclosed. [q-48]	Not Available	Not Available — Gemini Enterprise is closed-source; its internal code review processes are not publicly disclosed. [Google-wide]: Reviewers approve a CL once it definitely improves overall code health, even if not perfect. Non-mandatory polish comments are prefixed 'Nit:'. [Google-wide]: Technical facts and data override personal preferences in code review decisions; on style matters, the official style guide is authoritative. [q-52] [Google-wide]: Code reviews should receive a first response within one business day. Large CLs should be split into smaller, reviewable units. [q-54] [Google-wide]: Code ownership enforced via OWNERS files at every level of the monorepo file hierarchy, providing granular ownership that scales across thousands of engineers. [q-55]	Not Available — The specific code review tooling used for Gemini Enterprise (e.g., internal Critique tool, Gerrit) is not publicly disclosed.	Not Available — Gemini Enterprise's code review team structure (number of required approvals, senior reviewer roles) is not publicly disclosed. [Google-wide]: OWNERS file holders have approval rights for their repository subtrees.		Bug incidents extracted from public release notes reveal 4 recurring engineering problem categories: 1. Security vulnerabilities 2. Connector data quality and configuration 3. Agent orchestration bugs 4. API naming and UI consistency Practice: Bug fixes, security patches, breaking changes, and known issues are all documented in public release notes. Critical security issues are fixed automatically server-side with no user action required. [q-59]	Not Available — Gemini Enterprise's internal issue tracking system (e.g., Buganizer, Jira) is not publicly disclosed.	Not Available — The composition of Gemini Enterprise's bug-fixing team is not publicly disclosed. {Stakeholders}	Not Available — Additional bug-solving context is not publicly disclosed.	Gemini Enterprise uses admin-controlled feature toggles for model availability, requiring administrator action to enable preview model tiers. (New or experimental AI models are turned off by default, and only an admin can turn them on.) [q-63] Backward compatibility is explicitly maintained when new platform capabilities are introduced. Existing agent configurations are preserved on the prior editor when new UI experiences are introduced for newly created agents. [q-64] Not Available — Gemini Enterprise's internal development methodology (Agile, Scrum, sprint cadence, team size) is not publicly disclosed. [Google-wide]: Trunk-based development with no long-lived dev branches. The One-Version Rule mandates that developers must never have a choice of which version of a component to depend upon. Work is committed in small increments against the trunk regularly. [q-66] [Google-wide]: Large-scale automated changes across the monorepo executed using Rosie; presubmit checks gate all changes before landing. [q-67]	Not Available — Gemini Enterprise's internal project management tools (Jira, internal sprint boards) are not publicly disclosed. [Google-wide, not confirmed Gemini Enterprise-specific]: Rosie (large-scale automated changes), Piper (internal centralized VCS), Blaze/Forge (build system).	Not Available — Gemini Enterprise's engineering team structure (tech leads, engineering managers, product managers) is not publicly disclosed.	Not Available — Additional details about Gemini Enterprise's development lifecycle (sprint length, retrospectives, dogfooding practices) are not publicly disclosed.	[Google-wide]: Bug reports and issues are open with full comment history visible to all engineers (excluding customer PII), enabling cross-team learning from integration failures. [q-22] [Google-wide]: A unified test reporting system is available so any engineer can look up any build or test run, including all logs, supporting cross-team debugging and collaboration. [q-24] [Google-wide]: Code review conflicts that cannot be resolved via comment threads are escalated to face-to-face or video discussions; results must be recorded as a comment on the CL for future readers. [q-72]	Not Available — Gemini Enterprise's internal communication tools (Google Chat, mailing lists, internal wikis) are not publicly disclosed.			Gemini Enterprise maintains structured public release notes documenting all production updates: new features, bug fixes, known issues, breaking changes, and deprecated functionality. Release notes are also available as Atom feed, RSS feed, and via BigQuery for programmatic access. [q-25] Official documentation is organized into structured sections: Overview, Concepts, How-to guides, API and reference, and Resources — supporting different user profiles (conceptual readers, task-oriented practitioners, API integrators). [q-26] New observability metrics are documented at release time with references to monitored resources and metrics pages, indicating documentation is released alongside each feature. [q-27] Not Available — Internal documentation practices (doc review processes, freshness policies, documentation ownership, technical writer workflows) are not publicly disclosed.	Google Cloud documentation platform hosts all official Gemini Enterprise documentation. Release notes distributed via Atom feed, RSS feed, and BigQuery export. [q-28]	Not Available — The team responsible for documentation (technical writers, developer advocates, product managers) for Gemini Enterprise is not publicly disclosed.	Not Available — Additional documentation practices (API documentation generation tools, doc coverage requirements, style guides) are not publicly disclosed.
Glean Agents	Glean Agents is a closed-source enterprise AI agent. It automates knowledge-work tasks such as engineering standup reports, sales prospecting, code review assistance, and employee onboarding by indexing and reasoning over company data from tools like Slack, GitHub, and Jira. Glean maintains ~35 public auxiliary repositories under the Gleanwork GitHub organization. Primary language: Glean's core Index Server is written in Java. Glean's public tooling repositories are predominantly TypeScript Glean ships customer-facing releases on a monthly 'Drops' cadence [q-1]	Glean engineers run structured evaluations on agents using curated evalsets (a set of user instructions and ideal responses) of ~20 real-world examples with ideal responses, graded on a small set of binary sub-metrics (completeness, personalization, tone, groundedness) under a single 'north star' metric. [q-2] They continuously sample real user queries and feedback (upvotes/downvotes) into the evalset to keep evaluations aligned with evolving usage. [q-2] The most impactful improvements to AI agents come from a structured feedback loop: refining instructions, structuring context, tuning model parameters, and selecting the right language models. [q-2] After each eval run, Glean engineers review failing examples for patterns, then apply fixes in a strict priority order: refine instructions first, then restructure context (point to specific sources, use sub-agents), then tune LLM parameters (e.g. temperature), and only swap the model last [q-3] Glean advises building evalsets around complex, multi-step tasks first, not simple happy-path queries, because users routinely push agents across multiple sources and reasoning steps, making these the highest-value validation cases. [q-4]	LLM-as-judge graders aligned to human SME feedback are used alongside continuously running platform-level metrics (accuracy, relevance, completeness, groundedness). [q-5]	subject-matter experts (e.g., the sales team for a sales-prospecting agent) [q-6]	Glean's published benchmarks show Agentic Engine 2 achieved 94% task completeness, a 21% query increase in testing [q-7] [q-9]	Glean built a custom dynamic-deployment mechanism that loads the server from Cloud Storage without restarting, enabling nightly internal rollouts, weekly customer rollouts, and instant rollback via configuration change. [q-38]	Not Available	Internal engineers [q-10]	Each release tag is isolated from other experiments and associated with its own errors, enabling strong versioning across simultaneously running code versions. [q-39] Glean's hot-swap mechanism is built on custom Java ClassLoaders and ObjectWeb ASM for class-package remapping, loading JAR files from Cloud Storage in seconds. The platform is deployed in single-tenant isolated environments on AWS, Azure, or GCP. [q-40]	GitHub is the code-review platform, augmented by Glean's own PR Review and Resolve GitHub PR Feedback agents. Glean's PR Review agents scan code changes, flag risk areas, aggregate unresolved comments, and produce prioritized checklists to keep review quality consistent as the team scales. [q-12]	GitHub actions [q-14]	Distributed engineering reviewers; agents support reviewers rather than replace them, with humans still resolving threads. [q-41]	Pull request reviews are identified as one of the most common sources of delay. As teams scale, review quality becomes uneven and feedback cycles lengthen. The automated agent helps reduce iteration times and improve code quality by gathering context from various sources and providing actionable insights. [q-42]	Public issues reveal five problem categories: 1. security/PII exposure in tool outputs, 2. package dependency leaks, 3. configuration gaps, 4. absent CI test sandboxes requiring live production credentials, and 5. schema or documentation inconsistencies causing API rejections. [q-14] [q-15] Glean tracks issues across multiple GitHub repositories with a focus on authentication, state management, dependency handling, and API synchronization problems. [q-16]	ObjectWeb ASM tooling and Cloud Storage handle JAR distribution; online config flags enable rollback. Enterprise memory uses reinforcement learning to grade agent runs and derive better tool-use strategies. [q-17]	Agentic engine, and platform engineering teams own bug-resolution paths. [q-18]	Not Available	Glean follows a fast-iteration, experiment-driven methodology: nightly internal rollouts, weekly customer rollouts, and monthly public 'Drops', with every agent run producing context traces that improve the platform's orchestration layer. [q-43]	Not Available	Engineering, Product, and PMM contributors co-author public engineering posts, indicating cross-functional ownership of engineering decisions. [q-19]	Glean intentionally avoids monolithic platform design, using a horizontally layered open agent stack (context, models, orchestration, interfaces, security) so each layer evolves independently. [q-20]	Glean's engineering team uses its own agents as customer-zero: standup agents summarize prior-day PRs and tickets; Glean shares engineering work via a public blog, LinkedIn, and YouTube channel. [q-44] [q-45]	Internal coordination uses Slack, GitHub, and Jira (all indexed by Glean); external channels include the engineering blog, LinkedIn, YouTube, and the Gleaniverse community forum. [q-21]	All team members participate in collaborative editing of team pages. Engineering teams, product teams, and customer success teams communicate cross-functionally. Meeting participants use AI note-taking tools to capture decisions and action items. [q-47]	Meetings often take up over 50% of the work week, and key decisions, priorities, and action items often get trapped in meetings or lost entirely. Team pages reduce time spent searching for basic organizational information and provide consistent organizational context. There is a culture of transparency and continuous learning throughout the organization. [q-48]	Glean engineers publish engineering blog posts covering deployment internals, evaluation methodology, and agent architecture [q-26]	A Launch Documentation agent assembles release notes from specs, tickets, and merged changes; each public blog post includes an auto-generated 'AI Summary by Glean' [q-23]	Engineering, Product Management, and Product Marketing contributors [q-24]	Not Available
AmazonQ Developer	Amazon Q Developer is an AI-powered software development coding agent by AWS that assists developers with code generation, debugging, code transformation, and answering technical questions through IDE integrations (VS Code, JetBrains), a CLI, and a conversational chat interface. The CLI component (aws/amazon-q-developer-cli) is open source, built primarily in Rust with 1,940 stars and 424 forks. The backend agent service ships independently through AWS. Evaluated on SWE-bench Verified (score improved from 25.6% to 38.8%). [q-1]	Developers evaluate the agent by running it against public coding benchmarks (SWE-bench Verified, SWT-bench Verified) and publicly publishing full step-by-step trajectories so reviewers can audit failures — treating transparent trajectory publication as a core reliability practice. Agent score improved from 25.6% → 38.8% on SWE-bench Verified. [q-2] The team built an internal text-IDE harness called 'textcode' as a deterministic evaluation surface: during offline testing, the agent interacts only through text-based tool calls (open/close file, find/replace, revert) so runs are reproducible across evaluation cycles. [q-3] Internal dogfooding is a deliberate reliability practice: the team measures adoption and effort saved inside Amazon's own engineering org, using real internal usage as a continuous integration test for agent quality. Amazon developers saved 4,500 developer years of effort and 450,000 hours waiting for technical answers. [q-4] For the CLI codebase, every PR is gated by standard Rust test hygiene — cargo clippy with warnings treated as errors, cargo test with llvm-cov coverage, nightly rustfmt — executed across an Ubuntu + macOS matrix. [q-5]	SWE-bench Verified and SWT-bench Verified for agent evaluation; Terminal-Bench GitHub Actions workflow targeting terminal-bench-core; internal 'textcode' text-IDE harness for deterministic offline evaluation; cargo test + cargo-llvm-cov for Rust unit/coverage; cargo-deny for supply-chain checks; clippy; typos-cli. [q-6]	AWS applied scientists design and run agent evaluations. AWS platform/CLI engineers maintain repository-level test and benchmark workflows; Internal Amazon developers act as dog fooders whose real usage feeds back into reliability signals. [q-7]	A dependency-graph memory block is used in the SageMaker Canvas variant for evaluation: The agent infers missing variables from prior steps instead of re-prompting, reducing stuck/failure paths. Each node in the dependency graph represents a problem variable (problem_type, evaluation_metric, target_column). [q-8]	CI runs exclusively on GitHub Actions against the OSS CLI repo, with separate isolated workflows for each concern: Rust lint/test/coverage (rust.yml), agent benchmark runs (terminal-bench.yaml), docs deploy (mdbook.yml), typo checks, and merge-conflict detection. [q-9] For benchmark CI requiring AWS resources, GitHub→AWS OIDC federation is used so workflows assume a short-lived IAM role and upload artifacts to S3 — no long-lived access keys live in repo secrets. [q-10] Service-side agent deployments are decoupled from the CLI release train: backend capability upgrades ship through AWS independently and are announced via 'What's New' posts; CLI ships ~4–5 tagged semver releases/month. [q-11]	GitHub Actions; Rust 1.84.0 toolchain; cargo; cargo-llvm-cov; cargo-deny; typos-cli; mdBook with GitHub Pages for docs; AWS OIDC federation for keyless AWS access; Amazon S3 for benchmark artifacts. [q-12]	Release and infrastructure ownership sits with AWS platform engineers on the Q Developer CLI team; external open-source contributors interact with CI indirectly through required status checks on their pull requests. [q-14]	An auto-updater channel (feed.json) is bumped on each tagged release so already-installed CLIs pull new versions — making release cut-over a deployment step, not a user-action step. [q-13]	External code review uses the standard GitHub pull-request workflow against main, with CONTRIBUTING.md asking contributors to file an issue first for non-trivial changes — a lightweight RFC-style gate in front of review. [q-15] Internally, the developers dogfood Amazon Q Developer as a first-line reviewer of their own code, described under a 'Permission to Play' cultural framing that rejects narrow output metrics (LOC, story points) in favor of flow and completion signals. [q-16]	GitHub pull-request review with required CI status checks on the OSS repo; Amazon Q Developer itself used as an AI first-line reviewer during internal development. [q-17]	Pull requests on the OSS repo are reviewed by AWS staff maintainers on the Q Developer CLI team; external open-source contributors are the PR authors under review. [q-18]		Practice: Bug reports are collected using a standard template (“q doctor”) that gathers key system info like CLI version, OS, login method, and logs so issues can be reproduced easily. If a change breaks the main branch, maintainers quickly revert it to keep the system stable instead of trying to fix it later. [q-19] They also use recorded failed test runs (SWE-bench cases) to study mistakes and improve the system in the future. [q-51]	GitHub Issues and GitHub Discussions on the OSS repo for functional bugs; AWS Security Vulnerability Reporting portal for security-class reports; `q doctor` built-in diagnostic command for structured bug intake. Internal Amazon ticketing systems are Not Disclosed. [q-21]	AWS staff maintainers who triage and fix issues; the AWS Security team for CVE-class reports; external open-source reporters who file issues; AWS applied scientists who ingest agent-trajectory failures back into evaluation. [q-23]	Not Disclosed — additional context such as SLAs for critical bug resolution, internal on-call rotation structure, or post-mortem practices are not publicly disclosed. {Others} [q-22]	Trunk-based, high-frequency release methodology (~4–5 tagged releases/month). Publicly rejects industrial-age output metrics (lines of code, story points) in favor of knowledge-work metrics like developer wait-time and flow interruption. [q-24] Large-scale internal dogfooding is treated as a first-class methodology: Amazon engineers are the first users of each agent version. An internal developer shipped a non-trivial Rust feature in 2 days using Q Developer without prior Rust experience. Amazon's entire engineering org saved 4,500 developer years. [q-25] Applied scientists and platform engineers co-author public architecture posts, reflecting a research-plus-engineering pairing during development rather than a research-to-engineering handoff. [q-26]	GitHub for source control and review; an internal Amazon Q Business instance that indexes millions of internal documents so engineers can query institutional knowledge inside the dev loop; AWS 'What's New' for externally coordinated release announcements. Specific Scrum/Kanban tools are Not Disclosed. [q-27]	Internal Amazon developer-experience team (ASBX) drives adoption; applied-science leads run agent research; platform/CLI engineers own implementation; product managers coordinate external communications. [q-28]	Permission to Play' is an explicit cultural directive: developers are encouraged to experiment across unfamiliar stacks using the agent, and this experimentation is framed as a first-class methodology rather than personal time. [q-29]	Publicly observable developer communication happens on GitHub (issues, PR review threads, Discussions) and through long-form engineering write-ups on AWS DevOps and Amazon Science blogs; team alignment also surfaces externally through re:Invent conference sessions (DOP214: 'Unleashing generative AI: Amazon's journey with Amazon Q Developer'). [q-30] An internal Amazon Q Business instance indexes the company's internal knowledge repositories so engineers can ask questions of institutional documents directly — reducing synchronous ping-a-human communication during development. [q-31]	GitHub (issues, Discussions, PR review threads); AWS blog platform; AWS re:Post; YouTube (AWS Events channel) for re:Invent sessions; builder.aws.com for engineering narratives; internal Amazon Q Business instance for knowledge search. Internal chat/ticketing tools (Slack/Chime/etc.) are Not Disclosed. [q-32]	Communication stakeholders include AWS platform/CLI engineers, AWS applied scientists, AWS product managers coordinating external messaging, AWS developer-advocacy presenters, and external open-source contributors participating in GitHub discussions. [q-52]	Internal day-to-day chat, paging, and standup tooling is not disclosed in any public source. [q-53]	Three tiers of developer-facing documentation: (1) in-repo docs (README, CONTRIBUTING, SECURITY, /docs) — contribution and security processes documented inline so external contributors have a version-controlled reference; (2) an mdBook site automatically built and deployed to GitHub Pages via mdbook.yml on every push to main; (3) long-form architecture narratives published as AWS DevOps and Amazon Science blog posts. [q-33] Architectural decisions and agent-design rationale are treated as publishable documentation: design choices (textcode framework, memory-block dependency graph, multi-candidate generate-and-select) are written up in engineering blog posts that double as internal-to-external design records. [q-34] Contribution and security processes are documented inline in the repo (CONTRIBUTING.md, SECURITY.md) as authoritative, version-controlled references. [q-35]	Markdown in the repo; mdBook as the static-site generator; GitHub Pages for hosting; GitHub Actions (mdbook.yml) to automate deploys on every push to main; AWS blog platform and Amazon Science blog for long-form engineering posts. [q-36]	AWS platform/CLI engineers (in-repo docs and mdBook site); AWS applied scientists (architecture posts on Amazon Science blog); AWS technical writers for the customer-facing docs portal. [q-37]	Not Disclosed — additional internal documentation practices (doc review processes, doc coverage requirements, freshness policies) are not publicly disclosed. {Others} [q-38]

Agent Name	System Metric	Testing Methodologies				CI/CD Pipeline Practices				Code Review Process				Bug Solving Process				SE Methodology				Developer Communication				Hollistic Documentation			
	Description	Practices	Tools	Stakeholders	Others	practices	Tools	Stakeholders	Others	practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others
Gemini CLI	Google Gemini CLI is an open-source, terminal-based AI coding agent 250k+ loc 98% typescript 1000+ contributors stars: 102k+ high update frequency - 10 releases in 14 days (April 12–25, 2026) [q-8]	Gemini CLI engineers run a layered test suite with Vitest for unit tests and integration tests run on Docker/Podman sandboxes to check the functionality with the file system. Styling and Lints (Static Analysis tool) [q-1] In integration testing, run 5 times with the deflake script to confirm the implementation is not flaky. [q-4] Memory regression tests detect heap growth and leaks by comparing against committed baselines with a 10% tolerance; performance regression tests measure wall-clock time, CPU usage, and event loop delay with a 15% tolerance. Both suites run nightly in CI but are excluded from the default preflight check. [q-56]	Testing: Vitest. Linting/Formatting: ESLint, Prettier [q-7]	Core Gemini CLI maintainers, contributors at Google [q-7]	Not Available	GitHub Actions pipeline by the gemini-cli-robot bot. Generic CI/CD pipelines by GitHub Actions. maintainers approve cherry-picks for release branches [q-59] A daily schedule handles nightly releases; official patch/minor releases are triggered manually [q-59] The CI workflow runs on every push to a pull request and performs linting, the full suite of automated tests across macOS, Windows, and Linux on multiple Node.js versions [q-60] Finally, gemini-code-assist adds a "Summary of Changes, and code coverage comment. [q-60]	Github actions, gemini-cli-robot [q-62]	open source contributors, internal engineers [q-63]	Not available	Contributors submit small, atomic PRs that address single issues following Google's small CL philosophy. Provides an automated code review tool for finding anti-patterns, testing issues, and best practices. [q-66] Contributions require signing a Contributor License Agreement (CLA). (contributors.md) Reviewers should approve changes that improve the system's overall code health.	scripts/review.sh (AI-assisted review using Gemini) you can run the tool directly from the CLI prompt: /review-frontend <PR_NUMBER> [q-20]	Google-Gemini maintainers as code owners, open source contributors [q-72]		From the 10 issues analyzed, common engineering pitfalls include: (1) platform-specific regressions, (2) model fallback and quota handling failures, (3) parsing vulnerabilities with edge-case inputs (large text paste, code logs), (4) background process lifecycle management [q-77] The team uses bot-driven triage with /assign and /unassign commands allowing contributors to self-assign up to 3 issues concurrently. [q-75] External contributors who are assigned a 'help wanted' issue but have not opened a linked	GitHub issues, automated trage bot by Gemini [q-79]	Gemini-team maintainer [q-80]	Not Available	Trunk-based development following Google's philosophy of 'one repository, no dev branches' [q-82] The project uses a monorepo structure via npm workspaces [q-85] Four guiding principles published in ROADMAP.md 1. Power and Simplicity 2. Extensibility 3. Intelligent 4. Free and Open source	npm workspaces monorepo; semantic versioning with preview tags; conventional commit prefixes (fix, feat, chore); ESLint-enforced cross-package import restrictions; Apache-2.0 license header enforcement by ESLint. [q-88]	Open source contributors [q-89]	Not Available	Contributors self-assign issues using /assign and /unassign commands. Specific platform names not found like slack, discord. [q-37] [q-38] [q-39] [q-40] [q-41]	Core development team, community contributors. [q-42]	Core Google team, external maintainers, community contributors, and automated bots (gemini-cli-robot, triage bot, PR audit bot). [q-91]		The Google Cloud blog and Google AI for Developers blog publish architectural deep-dives. They maintain development documentations, pricing, token analysis, etc. Contributors must also update the relevant documentation in the /docs directory when a PR introduces a new command. Documentation follows the Google Developer Documentation Style Guide [q-93]	Markdown docs in-repo under /docs; official documentation site at geminicli.com; GEMINI.md for AI-agent context; release notes auto-generated and attached to each GitHub Release tag; Prettier for markdown formatting enforced by preflight. [q-47]	Google DevRel and the Gemini CLI engineering team, plus community PR authors who must update docs alongside any code changes they submit. [q-	Not Available
Kimi ok Computer	Kimi Ok Computer is a closed-source chat-based AI agent by Moonshot AI. It autonomously operates a cloud computing environment to complete complex multi-step tasks, including web browsing, code execution, image generation, deep research, etc Repo is closed source - but added related repository: kimi-agent-sdk Primary language: Python 50+ team member as contributors [q-49]	Internal reliability testing, benchmark - - Kimi code bench - kimi design bench - Claw bench [q-1] External benchmark performance - in-house evaluation framework adapted from SWE-agent [q-1] For the robustness check, they averaged over 10 independent checks [q-2] MoonshotAI employs systematic dogfooding as a reliability testing practice; ran 4k+ tool calls, over 12 hours of continuous execution as stress testing [q-3] Third-party enterprise evaluators independently validate model improvements using domain-specific benchmarks [q-4] Note: Mentioned comparison of the Kimi model, not the ok computer model	Internal proprietary benchmark suites, external industry-standard benchmarks (SWE-Bench series, Terminal-Bench, OSWorld, BrowseComp), and runtime output validators for document formats [q-5]	MoonshotAI RL infrastructure team, model development team, third-party enterprise evaluators [q-6]	When Agent Swarm mode is enabled on Ok Computer, parallel execution delivers a measurable lift on research-intensive retrieval tasks: BrowseComp rises from 83.2 (single-agent) to 86.3 (Agent Swarm), showing that horizontal scaling across sub-agents improves research breadth and accuracy beyond what a single agent can achieve. Benchmark reproducibility across these comparisons is controlled through versioned evaluation configurations—Claw Eval specifically runs at version 1.1 with max-tokens-per-step capped at 16,384. [q-7]	Before K2.6 was released externally, the RL infrastructure team ran it as a live autonomous agent managing real production operations — monitoring, incident response, and system operations — for five continuous days across 4,000 + tool calls. The anonymized worklog was published alongside the release announcement as the team's pre-release production validation record. [q-37] A dedicated translator.yml workflow automatically generates bilingual Chinese and English documentation on every docs change. [q-47]	Not Available	MoonshotAI model and infrastructure teams manage deployment internally; enterprise evaluators (CodeBuddy, Factory.ai, Vercel AI) publish independent pre-release benchmark results that function as external validation before general availability. [q-10]	Not Available	Not Available	Not Available	Not Available		When an agent encounters failure or stalls, the coordinator detects the interruption, automatically reassigns the task or regenerates subtasks, and actively manages the full lifecycle of deliverables—from initiation through validation to completion. [q-12]	Runtime fault-tolerance tools identified via community analysis: psutil for kernel process health monitoring, IPython KernelManager for lifecycle management and restart, and Playwright with Chrome DevTools Protocol for browser crash recovery. [q-13]	Bug resolution is handled internally by MoonshotAI; the RL infrastructure team's 5-day pre-release autonomous agent run generates operational failure signals ahead of external release. [q-14]	Not Available	MoonshotAI practices systematic dogfooding as a development methodology. The marketing team uses Claw Groups (Agent Swarm research preview) to run end-to-end content production campaigns, RL infrastructure team uses K2.6-backed agents for 24/7 autonomous operations — feeding real-world reliability and workflow data back into product development [q-15]	Not Available	Methodology stakeholders include the RL infrastructure team (autonomous operations and PARL training), the marketing team (dogfooding Claw Groups for content production), the broader Kimi Team research contributors, and third-party developers building agentic workflows on the agent SDK. [q-18]	Not Available	MoonshotAI maintains multiple public community communication channels: GitHub Discussions on kimi-cli for technical Q&A, a Discord server (discord.gg/FBDhVhVH7M) for real-time chat, a community forum at forum.moonshot.ai for longer-form discussions, and a Reddit community at r/kimi for broader community engagement. [q-19] Official product announcements and engineering deep-dives are published on kimi.com/blog with each major model and product release, including full benchmark tables, architecture descriptions, and third-party evaluator testimonials. MoonshotAI uses X (@Kimi_Moonshot) and Instagram (@kimiai.official) for social media outreach. [q-20] With each major model release, MoonshotAI publishes an anonymized operational worklog alongside the announcement—the K2.6 release included a redacted 5-day autonomous run record from the RL infrastructure team to let the broader engineering community audit the agent's real-world behavior. This practice of sharing edited-but-authentic operational records, rather than only synthetic benchmarks, functions as a transparency mechanism that bridges internal engineering decisions and public trust. [q-21]	GitHub discussion, Kimi-cli, discord, reddit [q-22]	Communication stakeholders span the MoonshotAI engineering and product teams, active open-source contributors, enterprise partners publishing public evaluations, and dual Chinese and international developer communities reflecting the product's global reach. [q-47]	Technical support for model deployment questions is routed to support@moonshot.cn per the Kimi K2.5 official repository. The platform.kimi.ai developer API provides an OpenAI/Anthropic-compatible endpoint for integration, with a Kimi Vendor Verifier (KVV) tool available to validate third-party provider deployment accuracy. [q-48]	MoonshotAI publishes detailed technical reports for each major model release, covering model architecture, training methodology, and Agent Swarm design, providing transparent academic-grade documentation of the model powering Ok Computer [q-23] [q-24] [q-25]	Documentation for Ok Computer is distributed through technical reports, the API developer portal at platform.moonshot.ai, and the community-maintained kimi-agent-internals reverse-engineering repository. [q-26]	Documentation stakeholders include the Moonshot AI research team, community analysts who independently document Ok Computer's internal architecture, and the broader developer community consuming API and integration documentation. [q-27]	Agent behavior documentation for Kimi Ok Computer is embedded in the SKILL.md files themselves. SKILL.md files serve simultaneously as runtime instructions for the agent and as behavioral documentation for the system, making the documentation executable. The community analyst who reverse-engineered this architecture captured the design principle: 'New capabilities come down to documentation. Write a thorough manual, put it in the right directory, and the agent is an expert in that domain.' This tight coupling means documentation quality directly determines capability quality—unusual in traditional software, where documentation and behavior are maintained independently. [q-28]
Anthropic Claude Code	Claude Code is an agentic coding CLI tool built by Anthropic that lives in the terminal, understands a full codebase, and autonomously executes tasks, including reading files and running commands, via natural language. It was created by Boris Cherny in September 2024 as a solo prototype and grew to a full product team by 2025, reaching GA with version 1.0.0 in May 2025. It is distributed as an npm package and also integrates with VS Code, JetBrains, a Desktop app, a web browser, and GitHub Actions. Loc and programming language: 512,000 lines of TypeScript stars: 124,000 Contributors: 51 Update Frequency: Claude Code ships approximately one external release per day, with roughly 60-100 internal (npm) releases pushed daily. Engineers averaged five pull requests per day during the summer of 2025. [q-1]	Anthropic uses a multi-layered eval strategy for Claude Code combining automated unit/end-to-end tests, model-based classifiers, dogfooding ('antfooding'), and A/B testing in production. [q-2] An evaluation ("eval") is a test for an AI system: give an AI an input, then apply grading logic to its output to measure success. The anthropic team adopts 2 types of evals - a) single-turn evaluations and b) Multi-turn evaluations. In a simple eval, an agent processes a prompt, and a grader checks if the output matches expectations. For a more complex multi-turn eval, a coding agent receives tools, a task, and an environment, executes an "agent loop" (tool calls and reasoning), and updates the environment with the implementation. Grading then uses unit tests to verify the working MCP server. [q-2] An evaluation suite is a collection of tasks designed to measure specific capabilities or behaviors. Tasks in a suite typically share a broad goal. [q-3] Anthropic's internal 'antfooding' practice—where Anthropic engineers ('ants') use Claude Code in daily work—serves as a continuous dogfooding loop. [q-4]	The Claude Code GitHub Actions integration (anthropics/claude-code-action@v1) enables CI-level automated code review, test generation, and issue triage [q-6] [q-7]	Anthropic's product engineering team, data science team, and security engineering team. [q-8]	A quality regression in April 2026 was diagnosed using Claude's own Code Review tool with Opus 4.7 reviewing the buggy pull requests—'the Code Review tool with Opus 4.7 found the bug, while Opus 4.6 didn't.' This demonstrates using newer model versions to catch issues in code changes. [q-9]	Claude Code ships approximately one public release per day, with 60-100 internal npm package releases daily. [q-63]	The build uses Bun as the bundler, producing a single self-contained cli.js file with zero external dependencies [q-67] [q-68]	Deployment stakeholders include: Anthropic's product engineering team (release pipeline), infrastructure/DevOps for enterprise managed settings, and package ecosystem maintainers (Homebrew, WinGet). Cat Wu serves as founding product manager overseeing release coordination. [q-69]	The 32 build-time feature flags and 26 hidden slash commands in the codebase enable staged rollouts and controlled exposure of experimental features. Environment variables such as DISABLE_COMMAND_INJECTION_CHECK (labeled DANGEROUS) and CLAUDE_CODE_ABLATION_BASELINE (disables all safety features) provide engineering escape hatches for testing. [q-70]	Boris Cherny (CEO of Anthropic) systematized code review feedback early in his career: he tracked recurring review comments in a spreadsheet and converted patterns reaching 3-4 occurrences into automated lint rules. This practice of automating repeatable review criteria is now applied to Claude Code's own development. [q-71] At Anthropic, Claude Code performs initial pull request reviews via a /code-review command. Humans approve final merges. They run multiple specialized subagents in parallel during review: one checks style guidelines, another examines project history, another flags obvious bugs. This Writer/Reviewer parallel-agent pattern is also documented in official best practices. [q-72] [q-73]	GitHub's code review interface, the /code-review slash command within Claude Code, the claude-code-action GitHub Action for automated PR reviews, and the internal Code Review tool (built on Claude Opus 4.7) used for diagnostic code review of regressions. [q-74]	Code review stakeholders include engineers submitting PRs, Claude Code (automated first-pass review), and human approvers for final merge decisions. The team ships PRDs by prototype, not document: 'PRDs are dead on the Claude Code team: prototypes replaced them.' Boris builds 5-10+ functional prototypes daily using parallel Claude agents, iterating rapidly based on feedback rather than writing specifications first. [q-75]	The team ships PRDs by prototype, not document: 'PRDs are dead on the Claude Code team: prototypes replaced them.' Boris builds 5-10+ functional prototypes daily using parallel Claude agents, iterating rapidly based on feedback rather than writing specifications first. [q-76]	Anthropic uses GitHub Issues as the primary bug tracker for Claude Code, with a stale-issue-manager automation that closes issues inactive for 60 days. [q-77]	GitHub Issues for formal bug tracking. Claude Developers Discord for peer support. [q-79]	Bug resolution stakeholders include the Claude Code product engineering team, Anthropic's security engineering team, the data infrastructure team (for infrastructure incidents), and the broader developer community filing issues on GitHub. [q-80]	An April 2026 postmortem publicly documented three overlapping quality regressions (reasoning effort default change, thinking cache bug, verbosity prompt) that degraded output quality. Anthropic committed to ensuring 'a larger share of internal staff use the exact public build of Claude Code' to prevent configuration drift between internal and public versions masking bugs. [q-81]	The Claude Code team uses prototype-driven development rather than traditional PRDs. Boris Cherny builds 5-10+ functional prototypes per day using parallel Claude agents, iterating based on direct feedback. The tool itself was validated by giving engineers access to the filesystem and observing rapid organic adoption (20% day one, 50% by day five) as the product specification. [q-82] Anthropic uses a flat organizational structure with the uniform title 'Member of Technical Staff' (MTS) for all engineers. This eliminates role-based hierarchies and establishes the expectation that everyone contributes across product, design, infrastructure, and research domains. [q-83] The team is working at rapid pace, with around 5 releases per engineer each day. Prototyping is done surprisingly quickly. [q-84]	Development tools: Bun (build), TypeScript (language), React+Ink (terminal UI), Yoga (layout), git (version control), npm (distribution), GitHub (repository/issue tracking), VS Code, and Claude Code itself for ~90% of code written on the project. [q-86] [q-87]	Anthropic engineering leadership [q-88]	Migration discipline is a core engineering principle: 'always make sure that when you start a migration, you finish the migration.' Partial migrations confuse both humans and AI models, leading to inconsistent code states. Engineers spend over $1,000 monthly on AI-assisted code migrations, using parallel subagents to execute large-scale refactors. [q-89]	Claude has become 'the first stop for questions that once went to colleagues. [q-36] [q-37] [q-38]	External community communication is managed through Claude Developers Discord and GitHub Issues. The Claude Code GitHub page has over 5,000 open issues. Slack (internal, inferred from feedback channel references), GitHub Issues (external bug tracker), Claude Developers Discord (community support), and the /bug in-product command for direct issue reporting [q-39]	Communication stakeholders include: the core Claude Code team (10+ engineers), the broader Anthropic engineering organization (antfooding users), external developer community on Discord and GitHub, enterprise customers, and partner integrators (VS Code marketplace, JetBrains Marketplace). [q-90]	Parallel agent architecture changes how engineers communicate work: Boris Cherny runs 5 instances of Claude Code simultaneously in terminal tabs with separate git checkouts, using system notifications to know when any Claude needs input. This shifts the communication model from deep serial focus to rapid context-switching across multiple parallel workstreams. [q-91]	Claude Code uses CLAUDE.md as its primary persistent documentation layer. Anthropic recommends keeping CLAUDE.md short [q-40] [q-41] [q-42] [q-43]	CLAUDE.md files [q-44]	individual engineers (CLAUDE.local.md personal notes), project teams (shared CLAUDE.md), IT/DevOps administrators (managed policy CLAUDE.md), and Anthropic's technical writing team [q-45]	Block-level HTML comments in CLAUDE.md files are stripped before injection into context, enabling maintainer notes that consume no context tokens. LLM-oriented code comments are written throughout the codebase 'for AI agents working on the codebase rather than human readers,' representing a shift in documentation audience. [q-46]
Google Gemini	Google Gemini is a family of closed-source multimodal large language models (LLMs) developed by Google DeepMind, powering the Gemini conversational AI chatbot accessible at gemini.google.com. It processes text, images, audio, video, and code, and serves as the foundation model across Google AI products and APIs. Deployed via Vertex AI, Gemini API, and Google AI Studio. [q-31]	Google DeepMind evaluates Gemini with a multi-stage benchmark and red-team pipeline: pre-training capability evals (MMLU, HumanEval), post-training safety/responsibility evals, and dogfooding inside Google products. Each Gemini release is accompanied by a model card and a tech report listing eval suites and scores. [q-18] Model impact assessments are conducted by the Google DeepMind Responsible Development and Innovation team, and are reviewed by the Google DeepMind Responsibility and Safety Council. {Quote 1, Summary 1}	Internal Google evaluation infrastructure; public benchmarks (MMLU, HumanEval, BIG-Bench-Hard, MATH); Vertex AI Evaluation Service for downstream agent eval; golden datasets curated per product surface. [q-2]	Google DeepMind research engineers, the Gemini model team, Google DeepMind Responsibility & Safety Council, Google Cloud Vertex AI team. [q-3]	Not Available — Additional evaluation details (HITL pipeline specifics, red-team methodology, benchmark pass/fail thresholds) are not publicly disclosed. {Others}	Gemini models are trained on TPUs using JAX and the Pathways system, then deployed via Vertex AI and the Gemini API with progressive rollouts. Model versions are tagged (e.g., gemini-2.5-pro-001, -002, -latest) and deprecated through documented lifecycles on the Google Cloud release notes page. [q-34]	TPU accelerators (v4/v5p/v6 Trillium), JAX, Pathways training system, Vertex AI Model Garden, Gemini API endpoints, A/B traffic splitters, Google Cloud release notes for version lifecycle tracking. [q-35]	DeepMind training infrastructure team, Google Cloud Vertex AI release engineers, the Gemini API product team. [q-36]	Not Available — Specific feature-flag configurations for Gemini model deployments are not publicly disclosed. {Others}	Internal to Google: Gemini engineers use Google's monorepo (google3) with mandatory pre-submit code review (Critique), readability certifications per language, and OWNERS files. No public PR review surface exists for the model itself. [q-38] Small CL Philosophy: Typical CL size is ~100 lines (1,000+ is too large). Each CL addresses one self-contained change, includes related tests, and can be independently reviewed. Refactorings must be separated from feature/bug-fix CLs. [q-39] Review Speed Targets: Reviewers must respond within one business day. LGTM-with-comments used for minor issues to avoid timezone delays. A typical CL should get multiple review rounds within a single day. [q-40] Review Focus Areas: Reviewers evaluate 8 dimensions — Design, Functionality, Complexity, Tests, Naming, Comments, Style, Documentation. [q-41] Progressive Improvement Standard: Approve if the CL improves overall code health, not only if perfect. 'Nit:' prefix marks non-blocking polish suggestions. [q-42] Mentoring Through Code Review: Reviewers leave educational 'Nit:' comments to help developers learn new language features, frameworks, or design principles — sharing knowledge is part of improving code health over time. [q-43]	Critique (Google internal code review tool), google3 monorepo, OWNERS files, language readability reviewers, presubmit triggers via TAP (Test Automation Platform). [q-44]	DeepMind and Google research engineers, language readability reviewers, ML infrastructure SREs. [q-45]		10 issues analyzed from googleapis/python-genai reveal 4 recurring engineering problem categories: Practice: Bug resolution follows a triage workflow routing issues to SDK maintainers (client libs), Vertex AI support (quota/infra), or DeepMind (model behavior). Persistent bugs require cross-team coordination. [q-47]	Google Issue Tracker (issuetracker.google.com, component 1166207), GitHub Issues on google-gemini/* and googleapis/python-genai, Google Cloud Status Dashboard for incidents, Google Cloud support cases. [q-49]	Vertex AI / Gemini API support engineers, DeepMind on-call rotation, googleapis SDK maintainers (python-genai repo). [q-50]	Not Available — Internal SLAs for critical bug resolution, on-call rotation structure, and post-mortem practices for the Gemini API team are not publicly disclosed. {Others}	Google engineering teams use structured weekly production meetings (30-60 min) to coordinate service development. Agenda items: upcoming production changes, metrics review, outage post-mortems, paging events analysis, nonpaging events, and prior action items follow-up. [q-52] Google engineering follows defined role structures: tech lead (TL) for technical direction and consensus-building, SRE manager (SRM) for performance management, and project manager (PM/TPM/PgM) for cross-functional coordination and deliverable tracking. [q-53]	google3 monorepo for code and documentation management (g3doc directories, Code Search integration); Markdown for documentation formatting; Google Docs for collaborative design documents requiring real-time multi-author editing. [q-54]	Not Available — Google DeepMind does not publicly disclose specific stakeholders involved in Gemini software engineering methodology beyond general team roles. {Stakeholders}	Not Available — Additional details about Gemini's development lifecycle (sprint length, retrospectives, specific dogfooding practices) are not publicly disclosed. {Others}	Google engineering teams use topic-driven group chats that are typically open access, allowing anyone to drop in and ask questions. Chats enable synchronous collaboration for quick problem-solving and informal technical discussions. [q-22] Google engineers extensively use mailing lists (topic-users@, topic-discuss@ Google Groups) for asynchronous technical discussions and decision-making. Any engineer can join or email these lists. [q-23] Most teams require an approved design document before starting any major project. Design docs outline architecture, implementation approach, and technical decisions, requiring stakeholder feedback and consensus before implementation begins. [q-24] Production meetings follow structured agendas: upcoming production changes, metrics, outage post-mortems, paging events, nonpaging events, and prior action items follow-up. [q-25]	Google Chat for real-time topic-driven group communication; Google Groups mailing lists (topic-users@, topic-discuss@) for asynchronous discussions; google3 monorepo g3doc directories (accessible via Code Search) for technical documentation; Google Docs for collaborative design documents with real-time multi-author editing. [q-26]	Google engineering communication involves tech leads who provide technical direction and build consensus in teams, engineering managers who handle performance management and catchall responsibilities, project managers who coordinate cross-functional work and track deliverables, SRE teams who run production meetings and articulate service state to stakeholders, and engineers across the organization who participate in group chats, mailing list discussions, and design document reviews. [q-57]		Each Gemini release ships with: a tech report (arxiv), a model card (capability/eval/safety summaries), a Google Cloud release-notes entry, and updated developer docs at ai.google.dev. The DeepMind 'Responsible Development of Gemini' post documents red-teaming and assurance practices publicly. [q-27]	arxiv.org for tech reports; model cards on ai.google.dev; ai.google.dev/gemini-api/docs for developer documentation; Google Cloud Vertex AI release notes; DeepMind blog for research disclosures. [q-28]	Gemini research authors (tech reports, model cards), Google DevRel docs writers (developer documentation), model-card reviewers. [q-29]	Not Available — Internal documentation processes (doc review workflows, freshness policies, technical writer involvement, doc coverage requirements) are not publicly disclosed. {Others}
Browser Use	Browser Use is an open-source agentic system that makes websites accessible to AI agents. It connects large language models to a Chromium browser via CDP (Chromium DevTool Protocol), extracting the DOM (Document Object Model) and screenshots on each step, then executing LLM-decided actions until the task is complete. Available as a pip package, a CLI tool, and a cloud API. Programming Language: Python. Stars: 94.1 stars Contributors: 318 Update Frequency: Avg, 7 releases per month [q-77]	Using Blacksmith runners on GitHub Actions, they achieve insane scaling, evaluating the agent on 100 complex web tasks in parallel in under 5 minutes end-to-end. They built the LLM judge directly into the agent code itself, running after the agent returns done. They also have a real-time validation layer built into the system. [q-1] They sourced their evaluation tasks from LLM-labeled user spans rather than synthetic sites. [q-2] Statistical rigor is applied to evaluation: each task is run multiple times, and results are aggregated with statistical bootstrapping, because single-run deterministic tests are unsuitable. [q-3] Unit and integration tests are defined under tests/ci/ and executed in parallel, with isolated failure reporting for the core test suite. [q-4] They stream every token, prompt, and cost directly into ClickHouse via Laminar for observability	Github Actions, Blacksmith runners, gemini-2.5-flash as llm as judge, pytest for automated test suite, Laminar for observability [q-5] [q-6] [q-7]	Engineering team [q-8] [q-9]	Pitfall experience: Initially, they tried embedding and clustering the failed traces to find common issues. It was a complete failure—the clusters didn't represent actionable product improvements [q-10]	A three-workflow GitHub Actions pipeline covers the full delivery lifecycle: 1. lint.yml (syntax, style, types on every push), 2. test.yaml (matrix unit tests plus per-PR agent evaluation), and 3. publish.yml (PyPI release triggered on a new GitHub release tag). [q-71] They scaled their infrastructure in 2 patterns - Pattern 1: Single backend with agent loop and isolated terminal uses a sandbox (Docker) for tool-calling via HTTP. Pattern 2: Isolate the agent. The entire agent runs in a sandbox with zero secrets. It talks to the outside world through a control plane that holds all the credentials. They switched from pattern 1 to 2 for scalability, decoupling, and security. [q-74] The tradeoff is an extra network hop (control plane) on every operation and three services to deploy instead of one. In practice, the latency is noise compared to LLM response times, and the operational complexity is the kind that ops teams already know how to handle. Their cloud service receives task requests, stores them in Amazon S3, sends them to an SQS (Amazon's Queue Service) queue, and returns an HTTP 202 response. A Lambda worker (serverless process runner) then processes each queued task and runs the agent until completion. [q-72]	Github Actions, per-PR agent evaluation via Blacksmith runners. AWS SQS, Lambda, ALB, and S3 form the core cloud production stack. SQS decouples ingestion from execution; S3 stores agent checkpoints, screenshots, and output files; The DLQ serves as a failure incident tracker [q-75] [q-76] [q-77] [q-78]	The cloud infrastructure team, Python developers and open-source contributors [q-79] [q-80]	Not Applicable [q-81]	All contributors must run pre-commit before submitting a PR. The pre-commit pipeline enforces check format (with auto-fix), Pyright type checking, codespell spell-check, and a suite of file integrity checks. [q-82] Pull requests automatically trigger the evaluate-tasks in the GitHub Actions job, which posts the evaluation score as a PR comment to assess whether a change improves or regresses agent performance. [q-83]	Github, Ruff - for formatting AI coding agents (e.g., Claude Code, Cursor) are explicitly supported as contributors via AGENTS.md, which provides embedded documentation and coding rules they must follow when generating code for this repository. [q-85] [q-86]	Developers, open-source contributors [q-87] [q-88]	check-added-large-files hook (maxkb=600) prevents oversized assets from being committed. pyupgrade enforces Python 3.11+ syntax constructs uniformly across the codebase. [q-89]	GitHub Issues is the primary bug tracker with a dedicated 'bug' label. As of May 2026, 863 bug-labeled issues have been closed, and 24 remain open, reflecting an active triage and resolution process. [q-90] Engineering blog shares recurring bug patterns: race conditions in distributed state, async event loop misuse, memory leaks from non-singleton HTTP clients, and cascading config failures. [q-91]	Github issue with triage [q-93] [q-94] [q-95]	The engineering team, open source community [q-96] [q-97]	Bug solving story: Memory leak from non-singleton HTTP clients required seven commits in one day to fully resolve. [q-98]	Browser Use adopts a minimal-agent philosophy: 'An agent is just a for-loop of messages.' Their development story reveals that their original first agent was discarded and rebuilt from scratch after the team reverse-engineered Claude Code and Gemini CLI, discovering that fewer abstractions produce more reliable agents. [q-100] Fast iteration with systematic learning: the cloud infrastructure was built through 4,000+ commits in one year. Mistakes were shipped, analyzed, and fixed forward. The gap between a working demo and production reliability defined the team's engineering maturity curve. Agent frameworks fail not because models are weak, but because their action spaces are incomplete. Instead of defining every possible action up front, start from the opposite assumption: the model can do almost anything. Then restrict. Based on their development experiences, they state: "Give the LLM as much freedom as possible, then vibe-restrict based on evals." [q-101]	agent-sdk [q-104] [q-105]	product and engineering team [q-106] [q-107]	Key lesson: less abstraction, let the model decide and reason in every step, access them to raw data to reason, analyze, and based on the evaluated data, add evaluation constraints.	The engineering blog (browser-use.com/posts) functions as a long-form knowledge-sharing and incident retrospective channel, publishing detailed post-mortems and architecture deep-dives accessible to the broader community. [q-48] [q-49] [q-50]	Slack (main), Discord, GitHub Issues and Pull Requests Twitter/X, and LinkedIn [q-51] [q-52] [q-53]	The engineering team uses Slack as its internal communication layer for evaluation-driven decision-making — deciding model upgrades, prompt changes, and code fixes based on Claude-generated analysis. [q-109]	A public status page at status.browser-use.com provides real-time incident transparency for all users of the cloud service. [q-111]	They support plenty of documentation. docs.browser-use.com provides structured developer documentation covering quickstart, agent configuration (all parameters), browser configuration, tool customization, production deployment, and API references. It serves both local developers and cloud API consumers. [q-54]	PostHog collects anonymous usage to help the team understand library adoption patterns and prioritize bug fixes. [q-57] [q-58]	The documentation team maintains docs.browser-use.com and the engineering blog, ensuring end users, integrators, and contributors have up-to-date references and architectural context. [q-59] End users and developers are primary documentation consumers: quickstart guides enable a working agent in under 5 minutes, while parameter references support advanced customization without source-code reading. [q-60]	The MIT license and transparent telemetry opt-out policy together reflect an open-source trust stance: the library is free, the data collection is disclosed, and the telemetry code is publicly available for inspection. [q-61]

Agent Name	System Metric	Testing Methodologies				CI/CD Pipeline Practices				Code Review Process				Bug Solving Process				SE Methodology				Developer Communication				Hollistic Documentation			
	Description	Practices	Tools	Stakeholders	Others	practices	Tools	Stakeholders	Others	practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others	Practices	Tools	Stakeholders	Others
Opera Neon	Opera Neon is a closed-source, subscription-based AI agentic browser by Opera Software. Built on Chromium, it combines local in-browser automation (Neon Do), conversational AI (Neon Chat), and cloud-based creation (Neon Make) — letting users browse, act on the web, and build digital outputs directly from their browser. Its model-agnostic AI engine selects from OpenAI, Google, and specialized models depending on task type. Primary Language: C++ (Chromium-based browser engine) [q-1]	The Opera security team conducted hundreds of internal pen-tests on Opera Neon before and during Early Access. This pre-launch hardening cycle was designed to surface and address key vulnerabilities in the AI agentic components before wider user exposure. [q-2] Opera launched Opera Neon in an Early Access phase to gather real-world usage insights I and to invite the broader security community to help identify gaps [q-3] Opera runs a public bug bounty program via BugCrowd as a structural complement to internal testing. It allows independent security researchers to report vulnerabilities ethically and to the mutual benefit of all parties. [q-4]	No specific tool name found [q-6]	Internal Security Team, External security researchers [q-7]	Their thoughts on agentic testing - "Testing is ongoing and expected to expand: the security team explicitly states that securing AI systems is tricky and unpredictable, calling for wider and more extended testing as the product matures" [q-8]	Opera Neon uses a phased rollout approach: the Early Access launch released to a limited number of invited users first, with more invitations following. This limits exposure surface during the validation period before wider availability. [q-55] Neon Make executes creation tasks on cloud virtual machines hosted on European servers, which keeps creation workloads separate from the user's local browser state. Neon Do, by contrast, operates fully locally in the user's running browser session. [q-56]	The virtual machine employed by Opera Neon is hosted on our secure European servers [q-57]	Opera's engineering team and infrastructure team [q-58]	The prompt injection fix was confirmed functional by external researchers on October 21, 2025 — one day after deployment — demonstrating a full fix-verify loop within a 24-hour window. [q-59]	Opera's security team actively reviews AI component designs for safeguards against malicious injection, collaborating with product teams throughout development. Security review includes analysis of how prompts are constructed and how user context is separated from web content. [q-60] Cross-team review during security incidents brings security, product, and engineering teams together. They shared their critical incident in the blog post, all relevant internal teams were brought in to review the vulnerability and the proposed fix before it was deployed. [q-61]	Internal code review tooling is not publicly disclosed. Code review is supplemented by: pen-testing as a validation layer, BugCrowd for external security review [q-62]	Security team and product team [q-63]	Opera Neon is fully closed-source. No public code review is visible. The only externally observable code quality signals come from incident post-mortems, security blog posts, and the external bug bounty program. [q-64]	Opera uses BugCrowd as its public bug tracking channel for external vulnerability reports. The team performs routine manual reviews of auto-rejected tickets. This is how the prompt injection report was recovered after being auto-dismissed. [q-65] On confirmation of a valid vulnerability, Opera follows a rapid response process: immediate acknowledgment, internal investigation, PoC reproduction, fix development, and same-day deployment — followed by external verification before public disclosure. [q-66] Opera holds CNA (CVE Numbering Authority) membership to evaluate submitted vulnerabilities and formally assign CVEs where criteria are met [q-67]	Bug tracking uses BugCrowd (public bug bounty) [q-69]	The security team owns vulnerability triage, PoC reproduction, and fix validation. The product team is notified and involved in implementing fixes during active incidents. External security researchers contribute via BugCrowd. The MITRE CNA program provides formal vulnerability classification. [q-70]	Neon Make uses a multi-agent orchestration model: one agent acts as a conductor, decomposes tasks into smaller steps, and coordinates specialist agents to execute each step. When the cloud agent encounters errors, it attempts to self-correct autonomously. [q-71]	Opera Neon was developed iteratively over two years. It began as internal agentic AI research, debuted publicly as 'Browser Operator' in March 2025, launched as Opera Neon waitlist in May 2025, and shipped in Early Access in September 2025 with interface and safety mechanics improved between each stage. [q-72] Opera Neon is designed with a 'human-in-the-loop' approach, meaning it will generally pause and request user interaction when actions such as completing a transaction or downloading a file require confirmation. [q-73] Tasks are self-contained workspaces in Opera Neon that allow you to work on a specific project using multiple sources of reference...without affecting or accessing other open Tasks or any other information elsewhere in Opera Neon. [q-74]	Chromium as the browser base cloud VMs for Neon Make, DOM tree parsing for web interaction, the Cards system for user-defined prompt customization, and human-in-the-loop gating for sensitive actions [q-77]	The engineering team, security team, and product team collaborate on iterative feature development as stakeholder [q-78]	Neon Make uses a multi-agent orchestration model: one agent acts as a conductor, decomposes tasks into smaller steps, and coordinates specialist agents to execute each step. When the cloud agent encounters errors, it attempts to self-correct autonomously. [q-79]	Opera follows responsible disclosure with external researchers: after receiving a report, they coordinate directly with the researcher, ask for a temporary hold on public disclosure while investigating, and confirm fixes before the researcher goes public. [q-34] [q-35] [q-36]	External communication uses BugCrowd for vulnerability submissions and a dedicated security inquiry portal about the bug bounty program [q-37]	Internal communication flows between the security team, product team, and engineering team during incidents. External communication is with security researchers (via BugCrowd and direct contact) and the broader user community (via security blog). [q-80]	The October 2025 disclosure timeline shows the full communication cycle: Oct 14 (report submitted) → Oct 17 (auto-rejected) → Oct 20 (Opera reopens and contacts researcher, requests hold) → Oct 21 (fix confirmed by researcher) → Oct 23 (Opera publishes public blog). [q-81]	Opera maintains a dedicated security blog series tagged 'Agentic Browser Security', which covers the security model of Opera Neon in detail: safeguard mechanisms, incident reports, and forward-looking recommendations. This is proactive public documentation rather than reactive disclosure. [q-38] Opera's security.opera.com hosts formal documentation covering: vulnerability policy, bug bounty scope and process, CNA membership details, security advisories, and a transparency report which forms a comprehensive security governance reference. [q-39]	Documentation is spread across: blogs.opera.com/security (security blog), blogs.opera.com/news (feature and release blog), operaneon.com/faq (product FAQ), security.opera.com (formal vulnerability and policy documentation), and press.opera.com (press releases). [q-41]	security team , marketing team [q-42]	Opera publishes incident post-mortems within days of resolution. The October 2025 prompt injection blog was published October 23, 2025 — two days after the fix was confirmed. No public API documentation exists as Opera Neon is a closed commercial product. [q-43]
Alibaba Mobile Agent	Mobile-Agent (v3) is an open-source, cross-platform GUI automation agent developed by Tongyi Lab at Alibaba Group. Its core is GUI-Owl — a foundational vision-language model built on Qwen2.5-VL, wrapped in a four-role multi-agent framework (Manager, Worker, Reflector, Notetaker) that automates tasks on Android phones, PCs, and web browsers without requiring app-specific training. Primary Language: Python Stars: 8700 Contributors: 22 Update Frequency: Active — major v3 release in August 2025; last commit November 2025. [q-1]	The team evaluates GUI-Owl across ten public benchmarks covering grounding, single-step decision, question answering, and online interactive task completion. Benchmarks include ScreenSpot-v2, ScreenSpot-Pro, OSWorld-Verified, etc. [q-2] Agent behaviour is validated by running the full agent stack against AndroidWorld and OSWorld interactive environments. They launch live agent sessions against real or emulated devices and measure end-to-end task success rate. [q-3] The team relies on issue reports to detect runtime failures that would otherwise require live device testing. [q-4]	AndroidWorld and OSWorld as live benchmark environments [q-5]	Evaluation is run by the Tongyi Lab research team at Alibaba Group [q-6]	The runtime code includes error-handling patterns: a 5-attempt retry loop for screenshot capture, try/except around JSON parsing with ast.literal_eval as fallback for Python-style dict outputs (introduced after PR #272), and RuntimeError raises for failed LLM API calls. These are guards against runtime failures, not test assertions. [q-7]	The only automated GitHub Actions workflow is pages-build-deployment, which rebuilds the project's GitHub Pages static website on every push to main. As of May 2026 there have been 258 runs of this workflow. [q-42]	GitHub Actions is the sole CI tool that used for website hosting, not code CI. Model artifacts are distributed via HuggingFace and ModelScope [q-43]	The Tongyi Lab core team [q-44]	The absence of a CI pipeline means that community pull requests are not automatically validated before review. Merged PRs were reviewed and merged by core maintainers manually, with no automated check on code correctness, style, or compatibility. Users have filed issues requesting better version control of remotely downloaded resources, reflecting the gap left by the manual release process. [q-45]	The repository has only 2 active pull requests, which suggests most code changes are committed directly by the core team rather than going through a formal PR review workflow. There is no public CONTRIBUTING.md or documented code review policy. [q-46]	GitHub Pull Requests is the code review surface. [q-48]	22 contributors are listed on GitHub. The core team is from Tongyi Lab, Alibaba Group. [q-49]	With 183 open issues and only 2 open pull requests, the project operates as a research artifact rather than a product codebase. External contributions are minimal; the team responds to selected issues but rarely merges community patches. [q-50]	The core team (primarily maintainer junyangwang0410) responds to issues on a best-effort basis with no formal SLA. For critical code bugs — where a community member provides a precise code-level diagnosis — the maintainer pushes a direct fix commit referencing the issue and closes it, often within one to two days. Issue #265 (smart_resize factor mismatch) followed this exact path: diagnosis in issue thread, commit pushed with 'Fix the bug in #265' message, issue closed the same day. [q-51] For benchmark reproducibility issues — the second most common category — the resolution pattern is different. The maintainer responds with internal evaluation logs and trajectory files to help the user compare outputs. If the root cause turns out to be an outdated inference dependency (vLLM version), no code fix is applied; the reporter closes the issue themselves after upgrading their environment. Issues #203 and #238 both followed this path. [q-52] Issues are not triaged with labels, milestones, or assignees. There is no visible prioritisation system: all issues sit in a single unlabelled queue. The 183 open vs. 67 closed ratio (73% open) reflects selective triage — issues get closed when either a code fix is committed or a reporter self-resolves. Architectural and recurring concerns (coordinate precision in #182, #278; context overflow in #266) remain open indefinitely because they have no single-commit fix. [q-53] Active community members provide first-line support for common setup and configuration questions, reducing the maintainer's response burden. In issue #238, community member BUPTyj posted a correct vLLM API parameter workaround before the core team responded. This peer-to-peer support in issue threads is the project's de facto help desk, filling the gap left by the absence of a documentation site or Discord channel. [q-54]	"GitHub Issues is the only public bug tracker [q-55]	Bug reports come from the open-source community, researchers and developers. The core team responds to selected issues, primarily those affecting benchmark reproducibility or API usage. [q-56]	The most persistent class of unresolved bugs is coordinate precision across all versions: #182 (19 comments, still open), #278 (coordinate offset in thinking model), and the historic #265 (smart_resize mismatch, now fixed). Each arises from the same root cause — the gap between the model's training resolution and the user's device resolution at runtime — but each version introduces a new variant. The --coor_type CLI flag introduced in v3 is the engineering mitigation, but issues continue to surface. [q-57]	The team follows a research-gated, version-per-subdirectory development model. Each major iteration is developed as a self-contained module in its own directory and released alongside an academic paper. Each release is triggered by a research milestone rather than a sprint cycle or feature roadmap. [q-58]	Version control is managed via Git on GitHub (X-PLUG organization). Python is the primary language. The only project-management surface visible publicly is GitHub Issues and GitHub Pull Requests — no Jira, Linear, Notion, or similar tool is referenced anywhere in the repository. [q-60]	22 GitHub contributors [q-61]	The codebase supports both Android (via ADB) and HarmonyOS (via HDC) device backends through a runtime flag (--adb_path vs --hdc_path). HarmonyOS support was added in the last known commit (b763787, 7 months before May 2026), indicating the team extends platform support incrementally after the initial release rather than bundling all platforms at launch. [q-62]	Research findings are published before formal conference submission, giving the community early access. Each release is announced via GitHub README updates, with dated news entries for every version milestone. [q-28] The README is maintained in both English and Simplified Chinese, reflecting the team's audience across Chinese research institutions and the broader international community. [q-29]	Guithub Issues [q-30]	The Tongyi Lab core team communicates with the academic community through conference presentations and arXiv papers. External users reach the team via GitHub Issues. [q-63]	Demo videos for each supported platform (phone, PC, and web) are embedded in the GitHub README, giving external collaborators a practical visual reference for the agent's capabilities. [q-64]	Each version of Mobile-Agent is documented through its own subdirectory README and a corresponding paper. [q-31] [q-32]	Documentation is hosted on GitHub (README files) [q-33]	Tongyi Lab team [q-34]	The action space for mobile and desktop are each formally documented in a dedicated table inside the technical paper (Tables 9 and 10), specifying every allowed action (key, click, long_press, swipe, type, open, terminate, etc.) with plain-language descriptions. [q-35]
Perplexity Comet	Perplexity Comet is a closed-source, Chromium-based AI browser that unifies traditional web browsing with agentic task execution. It contains two distinct AI layers: the Comet Assistant for reading, summarizing, and answering questions; and the Comet Agent, which autonomously executes multi-step tasks such as booking appointments, drafting emails, and managing workflows. Perplexity's search engine powers Comet, and the product targets both individual users and enterprise teams who seek intelligent workflow automation. Primary Language: TypeScript and C++ are typical for Chromium-based browser development Update Frequency: Active - Weekly changelog updates; Comet iOS, Comet Enterprise launched March 2026, prior feature releases shipping monthly throughout 2025 [q-1]	Perplexity developed BrowseSafe-Bench, a public evaluation suite of 14,719 real-world-style examples for benchmarking AI browser agent security. The benchmark checks prompt injection attacks: attack type, injection strategy, and linguistic style (how the instruction is phrased). This three-axis taxonomy helps the security team target training data gaps and systematically harden detection models. [q-2] Their models are continuously updated on new attack vectors discovered through their bug bounty program, red team exercises, and real-world detection events, ensuring they evolve faster than the threat landscape. [q-3] BrowseSafe-Bench evaluation showed that attacks hidden in visible webpage text, such as footers and tables, were harder for models to detect than attacks hidden inside HTML comments or metadata. This unexpected result changed how the team designed their training data. They also discovered that adding only three harmless distractor elements reduced detection accuracy from over 90% to about 81%, showing that the models were sensitive to realistic webpage noise and distractions. [q-4]	BrowseSafe and BrowseSafe-Bench as their security benchmark [q-5]	The Perplexity Security Team. External security researchers participate via the Vulnerability Disclosure Program and private bug bounty program, contributing new attack patterns to the retraining pipeline [q-6]	BrowseSafe-Bench includes 11 attack types, 9 injection strategies [q-7]	Comet ships on a weekly release, with each release documented publicly in a rolling changelog. The team publishes a 'What We Shipped' entry every week, covering both individual and enterprise capabilities.[q-41]	Closed sourced [q-42]	The Perplexity engineering team owns the release pipeline and publishes the weekly changelog. The security team has a parallel ownership stake in the BrowseSafe model deployment cycle, coordinating classifier updates independently of the browser release schedule. [q-43]	Comet expanded from a desktop-only (macOS, Windows) product to iOS within nine months of launch. [q-44]	Perplexity's internal pull request workflows, approval requirements, and merge strategies are not publicly documented. [q-45]	No public information is available on the specific code review tools used (e.g., GitHub Pull Requests, Gerrit, Phabricator) or automated review tooling such as linters or static analysis configurations . [q-46]	No public information is available on who performs code reviews, who holds merge permissions, or how the review process is structured across the Perplexity engineering organization. [q-47]	The only indirect signal is the weekly shipping cadence visible in the public changelog. Consistent weekly releases across desktop, enterprise, and iOS targets suggest a development workflow with short integration cycles, though the underlying review and merge process is not disclosed. [q-48]	Perplexity uses a Vulnerability Disclosure Program (VDP) to track its bugs. Researchers submit findings through a formal submission form. Valid disclosures earn acknowledgment, rewards and invitations to a private bug bounty program. AI-specific issues such as hallucinations and model safety are explicitly out of scope. Security problems are continuously used to improve the classifier. Edge cases found in production, red team testing, and bug bounty reports are turned into synthetic training data. The security team then re-trains BrowseSafe to detect new attack patterns before they become common. [q-50]	Audit logs (available for organizations with 50+ Enterprise Pro seats) provide usage and activity tracking for enterprise incident investigation [q-52]	The Perplexity Security Team, research community [q-53]	Comet is closed-source with no public issue tracker. Security incidents and compatibility issues surface through external security research reports, help center FAQ updates, and changelog entries rather than through a public GitHub issue list. [q-54]	"Experience has taught us that security isn't a feature to bolt on after launch, but a foundational requirement that requires reimagining how malicious action is conceived and where those attacks will come from." {Quote 1} [q-55]	Agile, Scrum, Kanban, sprint structure, and internal project management tooling are not publicly disclosed. The weekly 'What We Shipped' changelog format is indirect evidence of a continuous delivery process. Chromium as base platform reduces engineering scope by providing a mature browser foundation, allowing the team to direct velocity toward the AI layer rather than low-level browser infrastructure. [q-56]	All blog posts and changelog entries are attributed collectively as 'Perplexity Team' or 'Perplexity Security Team'. [q-57]	Perplexity piloted Comet under an invite-only model before broad release. New users received a limited number of invites to share, creating a controlled rollout mechanism that limits blast radius from bugs or security issues at initial launch. [q-58]	Perplexity communicates product updates through a public weekly changelog, an engineering and product blog at perplexity.ai/hub/blog [q-26]	X/Twitter, LinkedIn, Instagram, Threads, YouTube [q-28]	End users receive product news via changelog and blog posts. Enterprise administrators interact through the admin dashboard and dedicated support email. External security researchers engage through the VDP. The broader developer community engages through Discord and the public API documentation. [q-60]	Internal team communication practices are not publicly disclosed. Comet's launch blog post and enterprise announcement blog post are authored collectively as 'Perplexity Team', with no individual author attribution, reflecting a team-first public communication style. [q-61]	Perplexity maintains a dedicated Comet Resources hub at perplexity.ai/comet/resources as the primary user-facing documentation source. [q-29] Security documentation receives dedicated treatment. The prompt injection defense architecture is publicly documented across two blog posts (the blog post on mitigating prompt injection and the BrowseSafe research post) [q-30] The Comet Data Privacy and Security FAQ documents data handling at a level of granularity suitable for enterprise procurement reviews [q-31]	Documentation tooling includes the Comet Resources article hub, Perplexity Help Center (intercom-based), the public Hub engineering blog, Perplexity Trust Center at trust.perplexity.ai, and Security Documentation at perplexity.ai/hub/security. Compliance certifications (SOC 2 Type II, GDPR, PCI) are documented in the Trust Center. [q-32]	Help center articles are authored by Perplexity support staff . Security documentation is published by the Perplexity Security Team. Research papers and technical blog posts are attributed to the Perplexity Team or Perplexity Research [q-33]	There is no public developer API or SDK documentation for Comet specifically. The Perplexity API documentation at docs.perplexity.ai covers the search API, not the browser agent. Comet's enterprise integration relies on Chromium policy documentation rather than a Comet-specific developer reference. [q-34]